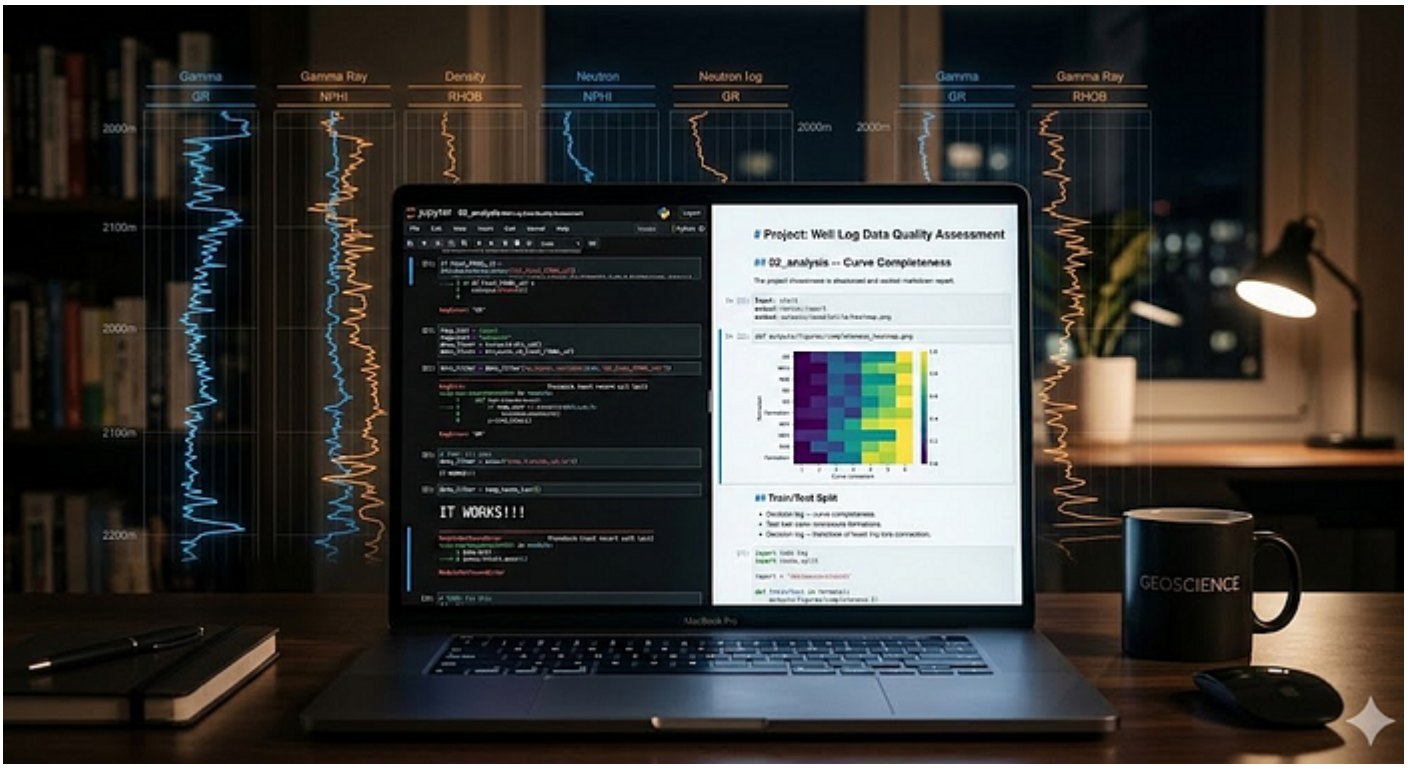


6 Jupyter Notebook Habits That Make Your Work Easy to Return To

From messy exploration to a record that still makes sense months later

Andy McDonald

Published March 7, 2026

Updated April 20, 2026

Free: No

Read on Freedium · original

Contents

Here is a situation working in Jupyter Notebooks that many people will recognise.

A colleague messages you asking how a particular calculation was carried out in a study that was worked on six months ago. You know the answer is somewhere in your jupyter notebook.

You open it, confident the logic and information is in there.

Forty minutes later, you are still not sure.

The code ran. The outputs exist. But the reasoning behind the key decisions is gone. Which Clay Volume cutoff did you use and why? Was normalisation applied to the gamma ray data before or after the formation tops were picked? There is a commented-out cell that might be relevant, or might be an abandoned experiment. You can't tell.

Then you start to notice that a cell further down the page has errored as a variable is not present. After much head scratching you find out it was calculated 5 cells further down.

This is not a messy notebook problem. Messy exploration is normal and expected. This is a documentation problem: the analysis produced a result, but it did not produce a record.

In subsurface work, that distinction carries real weight. Notebooks often sit at the base of interpretations that can feed future decisions, reserve estimates, and machine learning model training pipelines. When someone asks why a decision was made, "I think that was the approach" is not an audit trail. It is a gap.

These six habits don't slow down exploration. They ensure that what comes out of it is traceable, rerunnable, and readable by someone who was not in the room when it was written, including you, six months later.

This article does have a geoscience twist to it, but these habits can be applied to any data science project.

1. Treat the Notebook as a Decision Log, Not a Script

This is the shift that changes everything else.

Most notebooks are written as scripts: a sequence of code cells that produces an output. The problem with a script framing is that it records what happened, but not why. Code tells you that Gamma Ray values below zero were removed. It doesn't tell you whether that was a deliberate interpretation choice, a quality control step, or a temporary workaround that was never revisited.

A notebook written as a decision log treats every significant step as something that needs to be justified, not just executed.

The practical difference is in how you use markdown cells. Not as labels for the code below them, but as the reasoning that precedes it.

Compare these two approaches to the same step.

Label (not enough):

```
## GR Cleaning
Removing negative values and normalising.
```

Decision log (what you actually need):

```
## GR Curve: Cleaning Decisions
Negative GR values present in 3 wells (n=14 samples total). Spike pattern
and depth distribution consistent with tool noise rather than formation si
Excluded prior to any formation-level analysis.

Normalisation: min-max applied rather than z-score. Reason: preserves the
shape of the GR distribution across formations, which matters for the
cross-plot in section 4. Z-score would compress the tails where the
lithological contrast is most visible.
```

The code that follows executes the decision. The markdown records it.

When you return to this notebook in four months, you don't need to reverse-engineer the reasoning from the code. It is written down. When a colleague picks it up, they know not just what was done but why, and under what assumptions.

A useful test: if you cannot write a defensible markdown cell for a step, that is a signal the decision itself needs more thought before you commit it to code.

2. Write the Header Before You Write Any Code

At the start of any notebook, you can save yourself time later on by spending two minutes writing a header cell before writing a single line of code.

```
# Project: Volve Field -- Petrophysical QC
## Notebook: 03_formation_analysis -- Curve Completeness Assessment

**Purpose:**
Assess GR and RHOB curve completeness by formation across the Volve datase
Output used to determine viable curve inputs for the lithology classificat

**Input:**
`data/raw/volve_well_logs.csv`
Source: Volve open dataset, DISKOS export, retrieved 2026-01-15.
Wells in scope: 15/9-F-1, 15/9-F-4, 15/9-F-11 (producers only).

**Assumptions and exclusions:**
Intervals above casing shoe depth excluded (varies by well, see formation
GR values < 0 treated as noise and removed prior to analysis.
Brent Group subdivisions collapsed to group level due to sparse tops cover

**Output:**
`data/processed/curve_completeness_by_formation.csv`
```

`outputs/figures/completeness_heatmap.png`

Last updated: 2026-02-26 | **Status:** Complete

The two sections that matter most in geoscience or data science work are data provenance and assumptions.

Data provenance answers: where did this data come from, when was it obtained, and exactly which subset are you working with. A file path tells you nothing about whether the data came from a live database pull last week or a spreadsheet that was emailed across six months ago. Source and scope belong in the header because they are invisible everywhere else.

Additionally, a file path that resolves on your machine and nowhere else is not provenance. It is a dead end. If the data lives anywhere other than a shared location the next person is guaranteed to have access to, the header needs to explain how to get it:

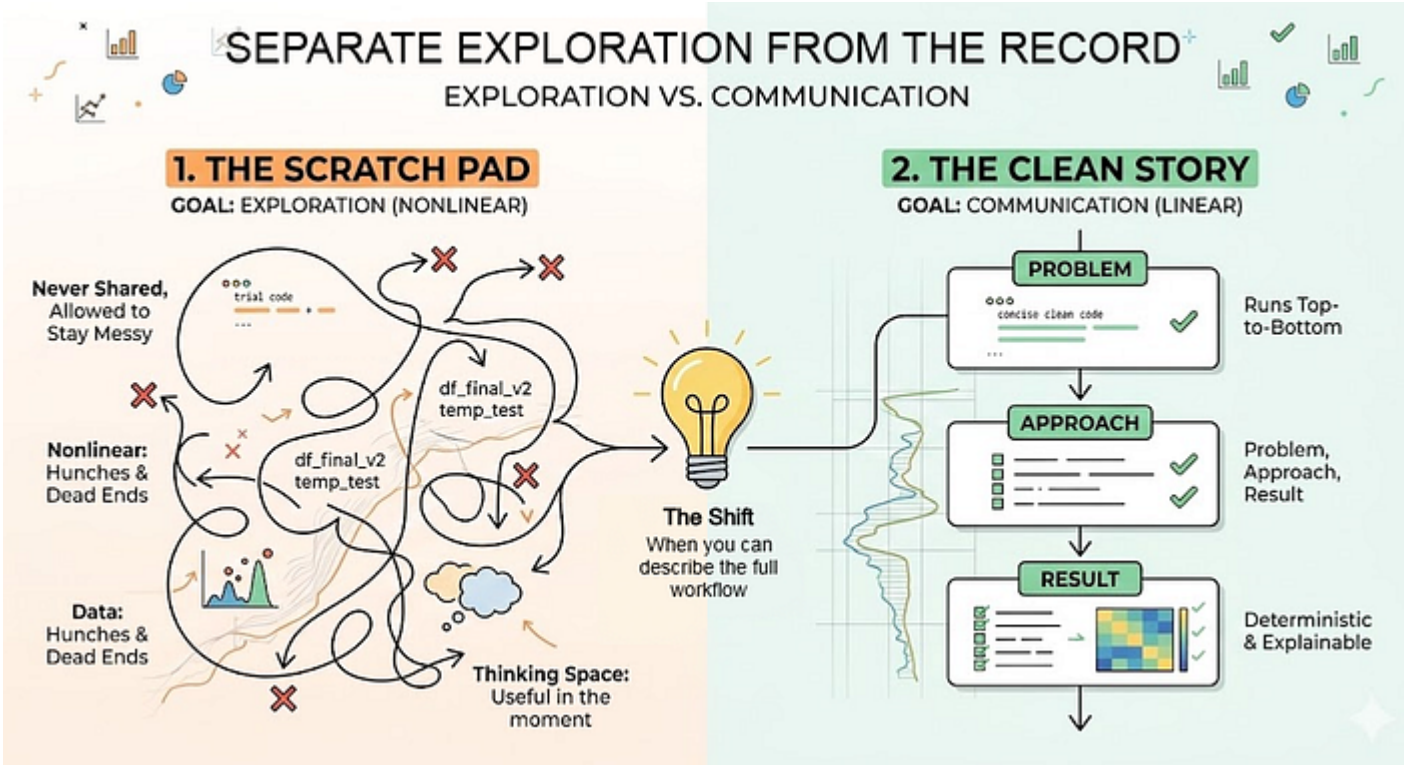
```
**Input:**
`data/raw/volve_well_logs.csv`
Source: Volve open dataset, DISKOS export, retrieved 2026-01-15.

Wells in scope: 15/9-F-1, 15/9-F-4, 15/9-F-11 (producers only).

Access: Shared project drive at \\server\projects\volve_qc\data\raw\
If unavailable, contact data management or re-export from DISKOS using
the query parameters documented in `docs/data_export_notes.md`.
```

The fields that matter when access is not guaranteed: where the file physically lives or can be retrieved from, who to contact if access is unavailable, how it was originally obtained so it can be re-obtained the same way, and any query parameters or export settings used. A re-export with different settings is not the same dataset, and that distinction will not be obvious to the person picking up the notebook six months later.

Assumptions and exclusions are where the interpretive choices live. Collapsing Brent Group subdivisions is a meaningful geological decision. Excluding above-casing intervals is a meaningful QC decision. Both are defensible. Neither shows up in the code without a comment, and comments are the first thing to disappear during a cleanup. The header cell is permanent.

A simple status message can inform you and other notebook users whether this is a complete notebook or if it is still a work in progress.

3. Separate Exploration From the Record



Exploration and documentation are not the same activity, and trying to do both in one notebook produces something that serves neither purpose well.

Exploration is nonlinear by nature. You are testing approaches, hitting dead ends, trying variations you will ultimately discard. That process should be unconstrained. A notebook full of experimental cells, abandoned branches, and intermediate print statements is not a problem. It's how analysis actually happens.

The problem comes when that exploratory notebook is also treated as the record. It looks like an analysis. It produces outputs. But the path through it is not a story. It is a session log.

The habit is to maintain two notebooks per project.

The working notebook is where exploration lives. It has no cleanliness requirement. Dead ends stay in. Commented-out experiments stay in. Variable names that only made sense at 2am stay in. It is never shared and never needs to be.

The record notebook is written once you understand the shape of the answer. It tells one story: problem, approach, result. It runs cleanly from top to bottom. Every significant decision has a markdown cell. Outputs are saved to predictable locations.

A practical signal for when to make the transition: when you can describe the approach out loud, in sequence, without referring to the notebook. If you can't do that yet, you are still likely in exploration territory.

The record notebook is not just a cleaned-up version of the working notebook. It should be written from scratch, with the benefit of already knowing what the analysis is and what functions or code snippets matter from your exploration notebook. That distinction matters. Cleaning up an exploratory notebook tends to preserve its structure while hiding its confusion. Writing the record from scratch produces something that actually reads as intended and can also highlight any issues in your initial analysis.

The record notebook matters even more as AI starts to enter the analysis workflow. A model can help generate interpretations, but someone still has to verify them. And that verification depends on having a clear record of what decisions were made and why. I've written about why verification becomes the hardest problem in technical AI work over on Substack. Link at the end.

4. Name Variables for the Person Who Comes Back

Variable naming is documentation that can't be separated from the code. Unlike comments, it can't be deleted. Unlike markdown cells, it is present on every line that references the object.

The goal isn't descriptive names for their own sake. It's names that carry two pieces of information: what the object contains, and what stage of the workflow it represents.

A stage-prefix convention makes both visible at a glance:

```
raw_logs            # loaded directly from file, unmodified
clean_logs          # nulls removed, depth range trimmed, curves validated
logs_with_vcl       # Vcl appended from a clay volume calculation
train_logs          # subset used for model fitting
test_logs           # held-out subset, not seen during training
```

For model objects, include the algorithm and the target variable:

```
model_rf_lithology        # random forest classifier, predicts lithology c
model_lgbm_permeability   # LightGBM regressor, predicts permeability (mD)
```

The rubric is simple: a name should tell you what it is and where it sits in the workflow. If you can't construct a name that does both, the object itself may not be clearly defined yet. That ambiguity is worth resolving before writing code that depends on it.

One habit that avoids the creeping `df2`, `df3`, `df_final` problem: name the object at creation, not at use. If you plan to rename it later, you will very unlikely rename it later.

5. The Kernel Restart Is Not Optional

Before closing any record notebook, restart the kernel and run all cells from top to bottom. If it doesn't complete cleanly, it is not finished.

The most common notebook failure mode is hidden state: a variable that was defined in an earlier session and no longer exists in the code, a file loaded from a path that no longer resolves, an import that worked once because another library happened to already be in memory. In the current session, everything appears correct. On a different machine, or six months later, it fails silently or not so silently at a cell with no obvious explanation.

The restart-and-run check is the only reliable way to surface these problems before they surface for someone else.

The practical objection is compute time. Some steps are expensive and re-running a full model training pass or reloading a large dataset on every review pass isn't realistic. The pattern that handles this cleanly is to cache intermediate outputs to `data/processed/` and load from cache in subsequent steps. The record notebook reads the cached output rather than re-executing the expensive step. The reproducibility standard still holds: the notebook runs top-to-bottom, deterministically, it just does not re-derive everything from raw data in a single pass.

The distinction worth being clear on: a notebook that produces the same outputs given the same inputs is reproducible, regardless of whether it uses cached intermediates. A notebook that only works when run in a specific order, with specific variables already in memory from a previous session, is neither reproducible nor fast. It is a future debugging problem waiting to surface at the worst possible moment.

6. Apply the Cold Open Test Before Closing

The first five habits are about writing the notebook. This one is about checking whether it actually works without you.

Before treating any record notebook as complete, try this: close it, open it fresh, and ask whether a colleague could follow it without asking you a single question. Not a full peer review. Just a five-minute scan. Can they identify the purpose from the header? Can they locate the key decisions in the markdown? Do they know what to do with the outputs?

If the answer to any of those is no, the notebook is not finished regardless of whether it runs cleanly.

In subsurface work this test has particular weight because notebooks routinely outlive the context in which they were written. A petrophysicist moves to another asset, goes on leave, or leaves the organisation. Six months later someone needs to understand what was done and why, and the only record is the notebook. It has to stand on its own.

A practical checklist to run before marking any record notebook complete:

```
## Notebook Sign-Off Checklist
[ ] Purpose is clear from the header without opening any code cells
[ ] Every significant decision has a markdown explanation, not just code
[ ] Data source and date of extraction are recorded in the header
[ ] Assumptions and exclusions are explicitly stated
```

```
[ ] Output file names make their contents obvious without opening them
[ ] Header accurately reflects what the notebook actually does
[ ] Kernel restarted and all cells run top-to-bottom without error
```

The last point on the header is easy to overlook. A header written at the start of a project often doesn't reflect what the notebook became by the end of it. A notebook titled "exploratory curve QC" that ended up producing the formation completeness dataset used in model training is misleading to anyone who encounters it later. Update the header when the scope changes. It takes thirty seconds and it is the first thing anyone reads.

The cold open test is the difference between a notebook that is technically complete and one that is actually usable. In a field where analysis is regularly revisited, audited, or handed over, that gap matters more than it might seem.

One Habit at a Time

These habits compound, but they don't all need to start at once.

If applying all six on your next project feels like too much overhead, start with the header cell. Write it before you write any code. It takes two minutes, it forces you to state the purpose and the assumptions before you have committed to any of them, and it is still there when you come back to the notebook in three months wondering what it was for.

The goal is not notebooks that look rigorous. It's notebooks that can be explained, re-run, and handed over without a conversation first. A header cell, honest markdown at each decision point, a clean top-to-bottom run, and sixty seconds asking whether a colleague could follow it cold gets you most of the way there without slowing down the exploration that precedes it.

These habits matter even more as AI starts to enter the analysis workflow. When a model generates an interpretation or a processed output, someone still needs to verify it. That verification depends on having a clear record: what data went in, what decisions shaped it, and what the output actually represents. A notebook without that record doesn't just make your own work harder to return to. It makes AI-assisted work harder to trust. I explored this problem directly in a recent piece on Substack — specifically why verification becomes the binding constraint in technical domains, and what that means for how you build and use AI tools.

→ The Jagged Frontier: Why AI Amazes and Frustrates in Equal Measure